Cross-Validation & Hyperparameter Tuning

A Beginner's Guide for AIML Students

Lovnish Verma | Project Engineer & Faculty, NIELIT Ropar

National Institute of Electronics and Information Technology, Ropar

What We'll Cover Today

01 Why a single train-test split is unreliable

02 K-Fold Cross-Validation — how it works

03 Stratified K-Fold for classification

04 What are Hyperparameters?

05 Grid Search CV — exhaustive tuning

06 Randomized Search CV — smarter tuning

07 Comparing results and choosing the best model

The Problem with a Single Split

What happens?

You split your data once — 80% train, 20% test.

The accuracy you get depends heavily on **which samples landed in the test set by chance.**

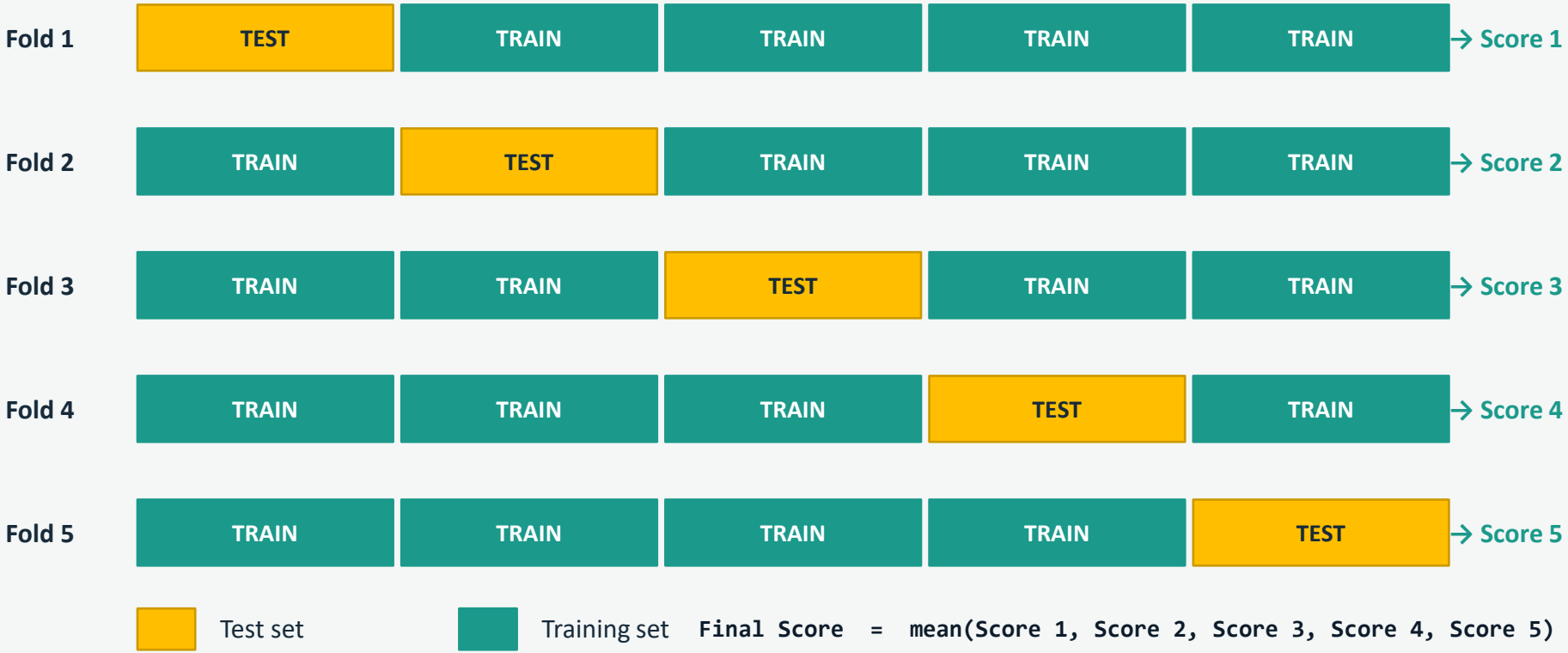
Different random splits → wildly different accuracies.

Same model, 5 different random splits:

⚠ **Results vary by ~6.7% — that's not reliable!**

K-Fold Cross-Validation

Split data into K equal parts. Each part gets a turn as the test set.



Stratified K-Fold — For Classification

Problem

Regular K-Fold splits data randomly.

If your dataset is imbalanced — e.g., 90% Class A and 10% Class B — some folds might have NO samples of the rare class.

This breaks training and evaluation.

Solution

Stratified K-Fold preserves class proportions in every fold.

If dataset is 50% A, 30% B, 20% C — every fold will also be ~50% A, ~30% B, ~20% C.

Always use this for classification tasks.

```
from sklearn.model_selection import StratifiedKFold
```

What Are Hyperparameters?

Parameters

Learned automatically from data during training.

Examples:

- Decision tree split thresholds
- Neural network weights
- Regression coefficients

Hyperparameters

Set by YOU before training begins.

Examples:

- n_estimators (number of trees)
- max_depth (tree depth limit)
- learning_rate
- C (regularization strength)

Common RandomForest Hyperparameters:

Hyperparameter	What It Controls	Typical Range
n_estimators	Number of trees in the forest	10 – 500
max_depth	How deep each tree grows	2 – 20, or None
min_samples_split	Min samples to split a node	2 – 20

Grid Search CV — Exhaustive Search

Grid Search tries EVERY combination of hyperparameter values you specify.

3 × 3 = 9 combinations, each evaluated with 5-fold CV

```
param_grid = {
  'n_estimators': [10, 50, 100],
  'max_depth': [None, 3, 5],
  'min_samples_split': [2, 5]
}

grid_search = GridSearchCV(
  estimator=model,
  param_grid=param_grid,
  cv=5,
  scoring='accuracy'
)
grid_search.fit(X, y)

print(grid_search.best_params_)
```

✓ **Guarantees finding the best combination** | ✗ **Slow for large grids**

- For **classification models** → it automatically uses **StratifiedKFold(n_splits=5, shuffle=False)**
- For **regression models** → it uses **KFold(n_splits=5, shuffle=False)**

Randomized Search CV — Smarter & Faster

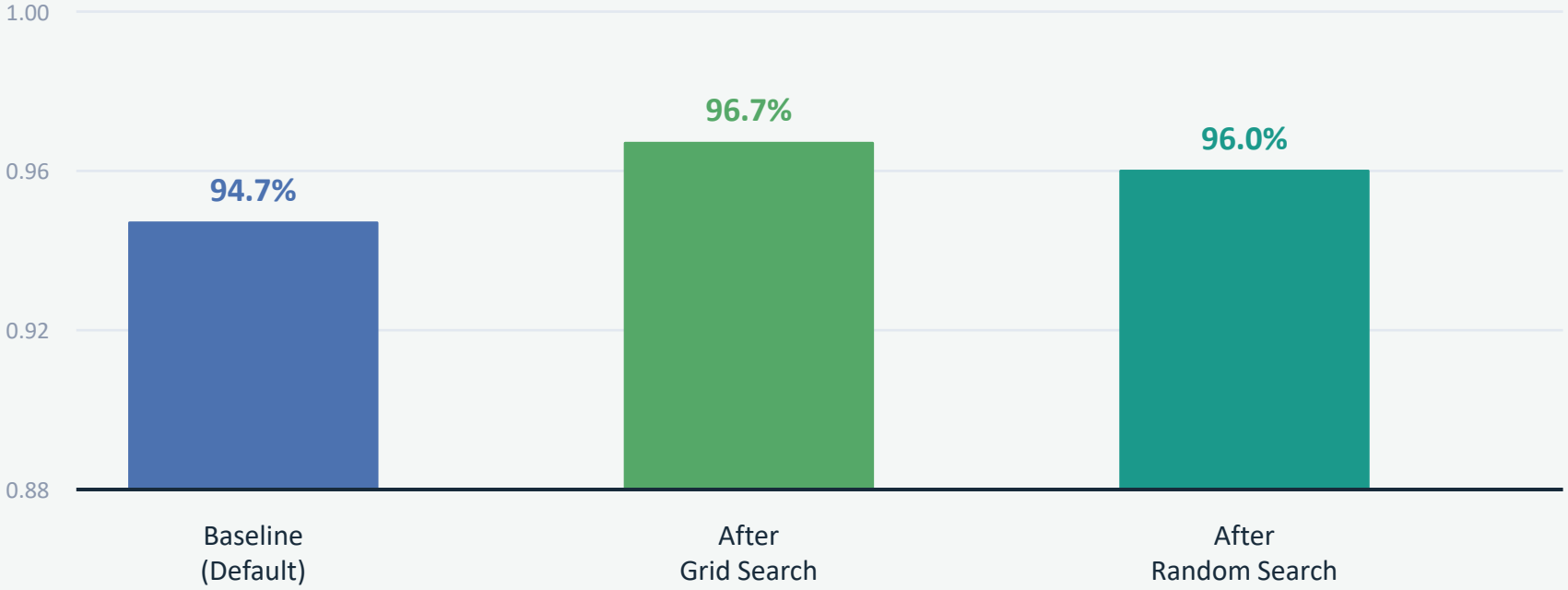
Instead of trying every combination, randomly sample N combinations from the parameter space.

Aspect	Grid Search	Randomized Search
Search strategy	Every combination	Random N combinations
Speed	Slow (large grids)	Fast (fixed n_iter)
Guarantees best?	Yes	Not always, but usually close
Continuous ranges	No — fixed lists only	Yes — use distributions
Best for	Small parameter grids	Large grids or limited compute

```
from scipy.stats import randint
param_dist = { 'n_estimators': randint(10, 200), 'max_depth': [None, 3, 5, 7], 'min_samples_split':
randint(2,15) }
random_search = RandomizedSearchCV(model, param_dist, n_iter=20, cv=5, scoring='accuracy',
random_state=42)
random_search.fit(X, y)   →   print(random_search.best_params_)
```

Rule of thumb: Start with Random Search → narrow range → fine-tune with Grid Search

Results — Baseline vs Tuned Models



Baseline

94.7%

Default hyperparameters
No tuning applied

Grid Search

96.7%

Exhaustive search
All 18 combinations tried

Random Search

96.0%

20 random iterations
Faster & close to optimal

Recommended Workflow

1

Load & Explore Data

Understand class distribution, missing values, and feature ranges.

2

Hold Out a Final Test Set

Set aside 20% as your final test set before ANY tuning begins.

3

Stratified K-Fold CV

Use StratifiedKFold on **training data** for reliable, stable evaluation.

4

Randomized Search

Run RandomizedSearchCV to quickly identify the best parameter region.

5

Grid Search (Fine-Tune)

Narrow down with GridSearchCV around the best region found above.

6

Evaluate Once on Test Set

Use best_estimator_ on the held-out test set. Report this score.

Quick Summary

Technique	What It Does	When to Use
K-Fold CV	Rotates test folds for a stable accuracy estimate	Always — instead of a single split
Stratified K-Fold	Ensures balanced class distribution in every fold	Classification (especially imbalanced data)
Grid Search CV	Tries every combination in the parameter grid	Small grids, thorough search
Randomized Search CV	Samples N random combinations from param space	Large grids, limited time or compute
best_estimator_	The trained model with the best parameters found	Final evaluation on held-out test set

```
Reliable Score  =  cross_val_score(best_estimator_, X_test)  →  Report this!
```

Thank You!

Key Takeaway:

Cross-validation gives you a trustworthy performance estimate.

Hyperparameter tuning finds the model configuration that performs best.

Combine both for production-quality machine learning.

Lovnish Verma | lovnishverma.in | @lovnishverma

NIELIT Ropar | Project Engineer | Faculty